Object Detection and OCR of Traffic Signs with Faster-R CNN and trOCR
Exploring Fine-Tuning with a Small Dataset

Anni Nieminen
Artificial Intelligence: Cognitive Systems
University of Gothenburg

Abstract

In this paper, we explore fine-tuning a Faster-R CNN model for object detection. Our main goal is to investigate the performance of the model in two scenarios: training with a very small dataset (consisting of Swedish traffic signs, data that we've collected and annotated partly ourselves), and a larger dataset (consisting of images of various Turkish road signs). Furthermore, we conduct a preliminary investigation on the performance of a base-trOCR-model on a small sample of our traffic sign image dataset containing text-based signs.

Specifically, we are interested in these questions:

- With fine-tuning, how does the Faster-R CNN model perform with both of our datasets in detecting and classifying images containing Swedish and Turkish traffic and parking signs?
- Are we able to reach satisfactory results when fine-tuning the model with very little data?
- What type of results do we get when we test base-trOCR's inference on text signs of our small dataset?

Our analysis shows that we do reach satisfactory results with fine-tuning the Faster-R CNN model with the very small dataset, and much improved results when fine-tuning the model with the larger dataset. Base-trOCR's inference, however, yields low quality outputs, indicating a need for further fine-tuning.

1 Introduction and background

Computer vision is a subfield of Artificial Intelligence having to do with extracting information from visual data (Marasinghe et al., 2024).

Image classification, in turn, can be seen as a base task for many subsequent tasks that have to do with computer vision, where the aim is to label an image with a single label. Object detection, however, deals with both recognizing as well as labelling multiple areas within a single image.

As for Optical Character Recognition (OCR), it is a task within the field of Computer Vision (CV) that deals with transforming various types of text, such as handwritten text or images of objects containing text, into machine-encoded text (Li et al., 2023).

1.1 Object Detection Models

R-CNN-models rely on a double task of performing both object detection (aka. region proposing), achieved with the Region Proposing Network, and object classification, achieved by the R-CNN, by classifying those detected regions. As illustrated in Figure 1, the model works by first applying the RPN network on the input image and extracting possible areas (or boxes) of interest. These are then sent to the classifier, and subsequently the localization of the bounding boxes is further post-processed. Figure 1 below further illustrates the R-CNN architecture.

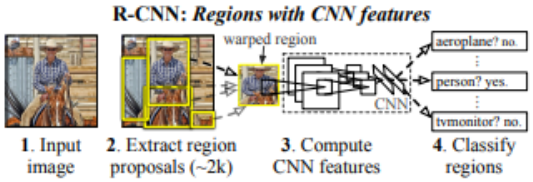

Figure 1: Illustration of the R-CNN architecture (extracted from Girshick et al. (2014), p.1)

The main difference between the base R-CNN model and the Faster version is the task of classifying the object proposals and refining the localization of the bounding boxes is combined (Girshick, 2015), thus slightly cutting down the training time. In other words, as the convolutional features are shared between the two sub-tasks (RPN and classification), the former works as "attention guide" for the latter (Ren et al., 2016).

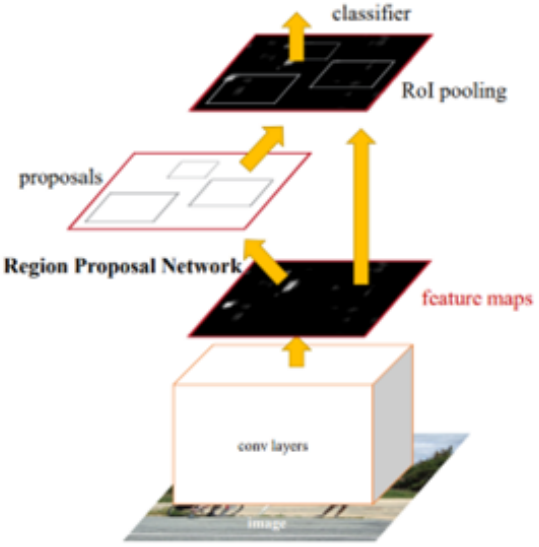

Figure 2: Illustration of the Faster R-CNN architecture (extracted from Ren. et al (2016), p. 3)

The Faster R-CNN model has been pre-trained with multiple different datasets, resulting in different model backbones. The model backbones, in essence, are convolutional networks work as feature extractors, identifying different shapes from the input images. In our own fine-tuning process, we use MobileNet V3 Large as the model backbone.

There exist also many other model architecture types for object detection tasks. One of the most known ones, YOLO (You Only Look Once), introduced by Redmon (2016) is known especially for its computational speed. Due to its unified network simultaneously predicting bounding boxes and doing the classification, the authors emphasize its ability to cut considerably down the image processing time.

Due to error analysis slightly favouring R-CNN in terms of bounding box localization and background detection accuracy (Redmon, 2016), choosing between the YOLO- and R-CNN-based models can thus be seen as a little bit of trade-off between time restrictions and the accuracy requirements. Furthermore, R-CNNs have been found to perform slightly better in detecting small objects (Tan et al., 2021).

1.2 OCR models

Many current Optical Character Recognition models still rely on using a CNN backbone for the text recognition and a RNN for text generation. Li et al. (2023), however, present trOCR, a transformer-based encoder-decoder model for text recognition, relying on processing the images as sequential patches, extracting their visual features and predicting the tokens based on the image and the already generated context. It is worth noting that the trOCR has been pretrained with textline images, referring to cropped text lines from various pdf-files. This means that trOCR can only be used for inference when the inputs are single lines of text.

As an additional exploration, we thus intend to explore trOCR's performance on a small subset of our data and perform a small-scale manual qualitative analysis of the results.

1.3 Image Augmentation

The training, or even fine-tuning, of computer vision models requires normally a large quantity of image data, especially if the model is expected to have a competitive performance (Xu et al., 2023). Various image augmentation techniques have risen in order to mitigate this challenge. Xu et al. (2023) introduce a hole taxonomy of various image augmentation approaches, but among the most common ones we can mention for instance cropping, adding contrast, blurring, rotating or flipping images, as well as altering the image's RBG-values. Making these augmentations both increases the size of the training set without having to manually collect more samples as well as helps the model to prevent overfitting.

2 Methodology

2.1 Data and Annotation Process

Traffic signs, and especially parking signs, may sometimes be tricky to interpret. This is often due to the multitude of additional text signs that accompany the "main" sign, thus altering its meaning. In Figure 3 below, we have a Swedish parking sign accompanied by three additional text signs. The first one indicates that one must pay a parking fee to park in this area from 9 a.m. to 5 p.m. on weekdays (as the additional times given inside parentheses would instead refer to days before *helg* (Sundays or public holidays)). Furthermore, the person parking in this area must have a blue or a red parking ticket. Moreover, the two additional signs indicate that parking is not allowed on Tuesdays between 9 a.m. and 2 p.m., and that people who live in *zone Va* have special parking rights in this area. This

goes to demonstrate just how much meaning these parking signs can convey!

Figure 3: An image from our small dataset.

Our first dataset consists of 192 images, consisting of images originating from a dataset from an open GitHub repository[1], our own screenshots taken from Google Maps, as well as our own photographs. The majority of the dataset contains images of informative signs (mainly parking-related), accompanied by supplementary signs, following the taxonomy defined in Fleyeh (2008). We performed an object detection annotation for the data with Label Studio. The traffic signs were annotated with bounding boxes, labelling traffic signs without any text with "traffic sign". Additionally, any lines of text in additional boards were labelled as "text signs". We then added meaning annotations to the traffic signs and transcribed the text on the objects that had been labelled as text signs. The meaning annotations refer to the Swedish names of these traffic signs (for instance *parkeringsplats* (parking area), *parkering förbjuden* (parking forbidden). Figure 3 below illustrates this annotation process.

In total, we have 22 classes of traffic signs in our dataset. The class imbalance is very high, with some classes containing only a few instances, and the largest class containing 98 images. Figure 3 above is an instance of class *"Parkeringsplats"*, accompanied by three text signs.

Figure 4: Screen capture of our annotation process with Label Studio.

Table 1 below lists the image augmentation settings we applied. For these, PyTorch's transform library was used. Figure 5 illustrates an augmented image.

Image augmentation type and parameters
Grayscale, 3
ColorJitter, brightness=0.5, contrast=0.5

Table 1: Image augmentation settings.

Our second dataset is downloaded from Kaggle[2], and it consists of 16 different classes of Turkish traffic signs. This dataset is considerably larger, training set consisting of around 13000 images and the test set holding 3000 images. It is also worth noting that most of these images have been collected from inside a vehicle and from taking screenshots from Google Maps, and consequently

[1] github.com/klintan/swe-parking-signs-groundtruth

[2] kaggle.com/datasets/nezahatkk/traffic-signs-in-turkiye

Figure 5: Augmented image from our dataset.

the target objects are often much smaller than in the first dataset. Figure 6 below shows an example image from this dataset.

Figure 6: Image from our second dataset (Turkish traffic signs).

2.2 Models and Hyperparameter settings

As for the models, for the object detection task we opted for Faster R-CNN model, with a MobileNet V3 Large backbone, with additional FPN (Feature Pyramid Network), which will help detect objects of different sizes. As for the pretrained weights, we chose *FasterRCNN MobileNet V3 Large FPN* Weights, which have been obtained from a comprehensive pretraining process on the COCO dataset.

During the data preprocessing stage, the pixel values of the images were normalized to values between 0 and 1, as required by the model. The images, however, were not resized, as it is not a requirement of the model.

During inference, the model returns outputs in the following format: List[Dict[Tensor]]. The dictionary holds boxes (coordinates of predicted objects), labels (predicted labels for each detected object) and scores, (the scores for each detection).

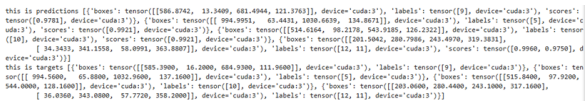

Figure 7: Screen capture of model outputs in inference mode.

We also explored multiple learning rates, finally settling upon 0.002, since we are working with pretrained weights. Furthermore, we used a Learning Rate Scheduler, which allows for a larger learning rate during the first epochs, and a lower one in the later epochs. Both the models were finetuned with a *fg iou thresh* of 0.6, indicating the minimum intersection over union (area of overlap / area of union) between the model's region proposal and the ground truth bounding box so that they are seen as positive during training of the classification head.

Hyperparameter and value
N of epochs, 30
Batch size, 4
Learning rate, 0.002
Learning rate scheduler step size, 7
Learning rate scheduler gamma, 0.1
Learning rate scheduler momentum, 0.9
Learning rate weight decay, 0.0005

Table 2: Model training hyperparameters (for first, small dataset consisting of Swedish traffic sign images)

Hyperparameter and value
N of epochs, 10
Batch size, 4
Learning rate, 0.002
Learning rate scheduler step size, 7
Learning rate scheduler gamma, 0.1
Learning rate scheduler momentum, 0.9
Learning rate weight decay, 0.0005

Table 3: Model training hyperparameters (for second, large dataset consisting of Turkish traffic sign images)

As for the OCR model, we chose Microsoft's

TrOCR-base. With this model, we did not perform any training, as our goal was simply to test its inference on a few random instances of our data.

3 Explorations and Results

During training, the Faster R-CNN calculates both Cross Entropy loss for the classification of the proposed regions as well as L2 regression loss of the location of the proposed regions. During the training of both of our models, we noted that the classification loss was decreasing somewhat quicker. We evaluated both the resulted models with Mean Average Precision (mAP), a metric often used to evaluate object detection models. mAP calculates the precision and recall over all the classes and returns the global mean average, considering the IoU results as well. More specifically, it calculates the average precision of multiple different IoU thresholds (for instance, starting with 0.5 as minimum IoU value for a prediction to be considered true positive, going all the way to 0.95).

$$\text{mAP} = \frac{1}{n}\sum_{i=1}^{n} AP_i$$

Figure 8: Mean Average Precision formula, where n is the number of classes and i is class.

Furthermore, we were interested in investigating the classification accuracy and IoU results independently from each other and have thus also added them in the evaluation. We will start by discussing the results of the fine-tuning process with the small dataset and move then to explain those of the larger dataset. We will also include a brief qualitative error analysis.

3.1 Results of Model Fine-tuning with Small Dataset

After having trained the model, we were pleasantly surprised by its performance on unseen data. Out of all the ground truth bounding boxes in testing data, 77% were correctly classified. Furthermore, the IoU reached 91%, indicating that the model was performing quite well with the task of predicting the object's locations on the images. As for The Mean Average Precision, the result was 0.369, indicating that the model is performing on a somewhat moderate level. mAP of larger objects (area > 96**2 pixels) was around 28% higher than that of small objects (area < 32**2 pixels). Mean average recall was approximately on the same level as precision.

Metric and value
mAP, 0.369
mAP small, 0.267
mAP large, 0.548
mAR, 0.373
IoU, 0.910
Clas. accuracy, 0.770

Table 4: Model performance (fine-tuned with small dataset).

What was the most surprising to us what that even with number of training images of less than 10, the model managed to predict those traffic signs correctly in the testing phase. Furthermore, the predictions of these instances had also high confidence scores. However, quite expectedly, when the model saw traffic signs with extremely low number instances in the dataset (some classes only had one or two training instances), it was not able to produce predictions. Figure 9, 10 and 11 illustrate predicted bounding boxes on unseen data, red bounding boxes illustrating the predictions and green ones ground truths.

Figure 9: Class 19, *verkningsområde slutar* (area of effect ends), only had 5 instances in the dataset. The model still managed to predict the label correctly.

Figure 10: Some images expectedly lacked predictions, due to the low number of training instances in the dataset.

3.2 Results of Model Fine-tuning with Larger Dataset

After having fine-tuned the model with the larger dataset and when testing its performance on unseen data, we reached a classification accuracy of 95%. The IoU result reached 84%, and mAP was around 66%. As with the first model, also this one performs better at detecting and classifying larger objects. Mean average recall is also again on approximately the same as mean average precision. Table 5 below summarizes the results on testing data.

Metric and value
mAP, 0.659
mAP small, 0.491
mAP large, 0.840
mAR, 0.663
IoU, 0.840
Clas. accuracy, 0.950

Table 5: Model performance (fine-tuned with larger dataset).

After having visually inspected the model's predictions on unseen data, we can confidently state that the model is performing well at detecting and classifying the traffic signs in the images. It is worth noting that a large proportion of the images in the dataset have been taken from a great distance, but this doesn't seem to disturb the model!

Figure 11: Model predicted two traffic signs correctly. One was left unlabelled.

Furthermore, after having investigated the model's output, we noted that the as the images often contained a rich variety of multiple objects in city settings, the model sometimes failed to detect all of the target objects. We believe that this is the main reason for the lower IoU score compared to the first dataset. Contrarily, compared to the first experiment with the small dataset, however, the model succeeded well in classifying the detected objects.

Figures 12, 13 and 14 below illustrate predicted bounding boxes on unseen data, red bounding boxes illustrating the predictions and green ones ground truths.

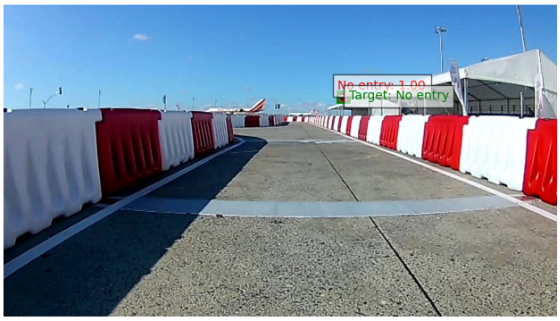

Figure 12: Model manages to detect and correctly classify even small objects.


```
Image 1:
Predicted Boxes: [[498.21918 174.88403 578.0409  263.54333]]
Predicted Labels: [15]
Predicted Scores: [0.99965143]
Target Boxes: [[497.92  173.808 579.84  266.832]
 [606.08  136.17  676.48  291.006]]
Target Labels: [15 13]
```

Figure 13: Model fails to detect the second object in the image.


```
Image 4:
Predicted Boxes: [[409.95303 164.12306 426.8134  212.91258]
 [224.2513  345.61536 241.3915  394.8987 ]]
Predicted Labels: [8 8]
Predicted Scores: [0.99566996 0.98745793]
Target Boxes: [[408.518 163.048 425.29  212.888]
 [849.382 396.584 868.55  437.88 ]
 [224.625 343.54  240.199 389.82 ]]
Target Labels: [8 8 8]
```

Figure 14: Model fails to detect the third traffic sign in the image.

3.2.1 OCR Expreriments with trOCR-base

Due to the time constraints of this project, we opted for simply testing the tr-OCR-base model's inference on our data, without any further fine-tuning. We conducted a few experiments with the annotations we had prepared with the first small dataset consisting of images of Swedish traffic signs. We randomly chose three images from the dataset and tested the model's inference on all the text in these images. We will discuss the results below in the captions of figures 15, 16, and 17.

Figure 15: This image has three different text signs, which contain in total 7 lines of text. Out of these, the OCR model was able to correctly recognize two lines, "8-21" and "Avgift". The rest of the text lines were only partially correctly recognized, as for instance "Alla dagar" was missing the the word "dagar" and "Övrig tid" was decoded as "Ovright".

4 Discussion

The Faster R-CNN model fine-tuned on the larger dataset achieved better results on all of the metrics apart from the IoU, compared to the first model which we fine-tuned with very little, yet augmented, data. As we already stated above, a manual inspection of the images in the second dataset showed that it contains much more detailed images and smaller objects compared to the first one, which might explain the difference.

As for the very preliminary explorations with testing the inference of trOCR, our results were not very promising. We must note that the model has not been explicitly pre-trained with Swedish data. However, with fine-tuning the model or exploring using Swedish LLMs in the decoder, as suggested by the authors in Li et al. (2023), we are confident that the performance of the model can be significantly improved.

Figure 16: The image contains even more text signs, five in total. The OCR model was able to recognize the numbers on the signs somewhat adequately, but made many mistakes with the words, for instance “reserverat” became “reserverate” and “besökare” became “besokane”.

Figure 17: With this image, the OCR model was able to correctly recognize “Tisdag”. As for the numbers, the model made some mistakes (such as recognizing the en dash between the hours as number 1).

5 Conclusion and future work

In this paper, we explored object detection of traffic signs with two different datasets. We explored different image augmentation techniques to compensate for the small size of one of our datasets. Additionally, we explored an trOCR model and briefly looked into the performance of the base model with our data. A side task was to familiarize ourselves with the ‘first steps’ of such pipeline, referring to the data collection and annotation.

Fine-tuning the Faster R-CNN model with a very small augmented dataset yielded mediocre results, whereas repeating the process using a considerably larger dataset improved the model’s performance.

This project can be refined and improved in many ways, which were unfortunately not possible due to our time and resource limitations. For instance, various other under-and oversampling methods and new data augmentation techniques could be explored in order to compensate for the low number of training instances. Fine-tuning the OCR model would also be crucial in order to reach better results.

Additionally, it could be interesting to explore other object detection models and use different types of datasets. Comparing the performance of transform-based CV models, such as the Visual Transformer on a task like traffic sign object detection might also be an interesting challenge! Lastly, we think that the annotations of our dataset, especially the transcribed text signs, can work as a base for further project ideas.

6 References

Fleyeh, H. (2008). Traffic and road sign recognition (Doctoral dissertation). Dalarna University.

Girshick, R. (2015). Fast R-CNN. 2015 IEEE International Conference on Computer Vision (ICCV), Santiago, Chile, 2015, pp. 1440-1448, doi: 10.1109/ICCV.2015.169.

Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 580-587).

Li, M., Lv, T., Chen, J et al. (2023, June). Trocr: Transformer-based optical character recognition with pre-trained models. In Proceedings of the AAAI Conference on Artificial Intelligence (Vol. 37, No. 11, pp. 13094-13102).

Lu Tan, Tianran Huangfu, Liyao Wu et al. (2021) Comparison of YOLO v3, Faster R-CNN, and SSD for Real-Time Pill Identification, https://doi.org/10.21203/rs.3.rs-668895/v1

Marasinghe, R., Yigitcanlar, T., Mayere, S et al. (2024). Computer vision applications for urban planning: A systematic review of opportunities and constraints. Sustainable Cities and Society, 100, 105047. https://doi.org/10.1016/j.scs.2023.105047

Redmon, J. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition.

Ren, S., He, K., Girshick, R et al. (2016). Faster R-CNN: Towards real-time object detection with region proposal networks. IEEE transactions on pattern analysis and machine intelligence, 39(6), 1137-1149.

Xu, M., Yoon, S., Fuentes, A et al. (2023). A comprehensive survey of image augmentation techniques for deep learning. Pattern Recognition, 137, 109347.

6.1 Appendices

The notebooks of this project can be found at https://github.com/Anurni/AICS_project_object_detection